ABACUS INFORMATION SYSTEMS, INC.

POWERSHELL 7

Installation and Migration Guide for Windows Administrators

Install the latest stable PowerShell 7 release alongside Windows PowerShell 5.1, confirm it works, test the tools that matter, and put a safe update process in place.

Document Type	**Technical Procedure**
Audience	**Windows administrators and support technicians**
Impact Level	**Low for installation; medium if scripts or scheduled tasks are migrated**
Validated PowerShell Release	**PowerShell 7.6.6 (validated September 2026)**
Version	**1.1**

IMPORTANT: PowerShell 7 does not replace or upgrade Windows PowerShell 5.1 in place. It installs side-by-side, uses pwsh.exe, and should be treated as a controlled migration of scripts and tools rather than a removal of 5.1.

1. PURPOSE AND SCOPE

This guide walks an IT administrator through a safe, repeatable PowerShell 7 deployment on supported Windows clients and servers. It covers the checks to run first, installation, validation, compatibility testing, updates, troubleshooting, and rollback. Windows PowerShell 5.1 stays in place.

RECOMMENDATION: Keep Windows PowerShell 5.1 available while you test production scripts, modules, RMM tasks, and scheduled jobs under PowerShell 7. Installing the new shell is the easy part. The real migration is proving the old automation still behaves.

2. BEFORE YOU BEGIN

2.1 Prerequisites

- Local administrator rights for MSI or machine-wide installation
- A supported Windows client or Windows Server operating system
- Internet access to the Microsoft package source or an approved internally staged installer
- A maintenance and test plan for scripts, profiles, modules, scheduled tasks, RMM jobs, and remoting workflows
- A backup or source-controlled copy of production scripts and profile files

2.2 Capture the current environment

```
$InventoryPath = "$env:USERPROFILE\Desktop\PowerShell-PreMigration-Inventory.txt"

& {
   $PSVersionTable
   "`nPSHOME:"
   $PSHOME
   "`nProfiles:"
   $PROFILE | Select-Object *Host*
   "`nModule paths:"
   $env:PSModulePath -split ";"
   "`nAvailable modules:"
   Get-Module -ListAvailable |
      Select-Object Name, Version, Path |
      Sort-Object Name, Version
} | Out-File -FilePath $InventoryPath -Width 300
```

FIELD NOTE: Run this inventory from Windows PowerShell 5.1 before installing anything, then save the output with the change record. If a module or profile acts differently later, you will have a clean before-and-after trail instead of a guessing contest.

3. CHOOSE THE INSTALLATION METHOD

Method	Best Use	Abacus Guidance
WinGet	Windows client computers	Best choice for a one-off client installation when WinGet is available.
MSI	Windows Server and managed enterprise deployment	Best choice for servers, RMM, Configuration Manager, and other controlled deployment workflows.
ZIP	Testing, side-loading, or special cases	Use when a non-installed copy or multiple PowerShell 7 versions are required. PATH and shortcuts may require manual work.

COMMON GOTCHA: WinGet is not available on Windows Server 2022 or earlier. Windows Server 2025 includes it only with Desktop Experience. In other words, do not build a server deployment plan around a command the target server does not have.

4. INSTALL POWERSHELL 7

Option A - Windows client using WinGet

Open an elevated Windows Terminal, Command Prompt, or Windows PowerShell session. First confirm the stable package, then install it:

```
winget search --id Microsoft.PowerShell --exact
winget install --id Microsoft.PowerShell --source winget --accept-source-agreements --accept-package-agreements
```

For environments that specifically require the MSI package format through WinGet:

```
winget install --id Microsoft.PowerShell --source winget --installer-type wix --accept-source-agreements --accept-package-agreements
```

ADMINISTRATIVE NOTE: Beginning with PowerShell 7.6, the standard WinGet command installs the MSIX package by default. MSIX is suitable for normal interactive use, but it is installed per user and does not support PowerShell remoting or certain system-level configuration changes. Use --installer-type wix when an MSI-based administrative installation is required.

Option B - Windows Server or managed enterprise deployment using MSI

Download the correct MSI for the system architecture from the official PowerShell release. Validate the source and file, stage it through the approved software deployment process, and install it under the organization's normal change controls. Example silent installation:

```
msiexec.exe /i PowerShell-7.6.6-win-x64.msi /qn ADD_PATH=1 REGISTER_MANIFEST=1 USE_MU=1 ENABLE_MU=1
```

VERSION NOTE: These instructions are validated for PowerShell 7.6.6. Microsoft has announced package-format changes beginning with PowerShell 7.7, including removal of the MSI package. Revalidate the available deployment packages before adapting this procedure to a later release.

Check installed PowerShell packages:

```
Get-AppxPackage Microsoft.PowerShell
Get-Package -Name "PowerShell*" -ErrorAction SilentlyContinue
```

SECURITY NOTE: Do not enable PowerShell remoting during installation unless it is part of a separately approved remoting design. Installing PowerShell 7 is one change. Opening remote administration paths is another.

5. VALIDATE THE INSTALLATION

Close any open terminals, start a fresh session, and launch PowerShell 7 with pwsh. Confirm the edition, version, executable path, architecture, and installation type:

```
pwsh
$PSVersionTable
$PSHOME
[Environment]::Is64BitProcess
(Get-Process -Id $PID).Path
Get-Command pwsh | Select-Object Source, Version
```

Expected results:

- PSEdition reports Core
- PSVersion reports 7.6.6 or a later approved stable 7.x release

- PSHOME normally points to C:\Program Files\PowerShell\7 for an MSI installation
- The process path resolves to pwsh.exe, not powershell.exe
- Record whether the deployment is MSI, MSIX, or ZIP so future updates and removal use the same installation method.

FIELD NOTE: Windows PowerShell ISE runs Windows PowerShell 5.1 only. Installing PowerShell 7 does not upgrade ISE. Use Windows Terminal, the PowerShell 7 console, or Visual Studio Code for PowerShell 7 work.

STOP CONDITION: If pwsh opens the wrong version, the executable path is unexpected, or the architecture does not match the deployment plan, stop before migrating scripts. Fix the installation or PATH issue first. There is no prize for carrying a bad foundation into production.

6. TEST MODULES, PROFILES, AND SCRIPTS

6.1 Understand the boundary

Windows PowerShell 5.1 runs as powershell.exe and reports the Desktop edition. PowerShell 7 runs as pwsh.exe and reports the Core edition. They also use separate profile locations. PowerShell 7 can discover Windows PowerShell module paths, but seeing a module is not the same as proving it works.

```
$PROFILE | Select-Object *Host*
$env:PSModulePath -split ';'
Get-Module -ListAvailable | Sort-Object Name, Version
```

Example scheduled task configuration:

```
Program/script:
C:\Program Files\PowerShell\7\pwsh.exe

Arguments:
-NoLogo -NoProfile -NonInteractive -ExecutionPolicy Bypass -File "C:\Scripts\Example.ps1"

Start in:
C:\Scripts
```

FIELD NOTE: Use the validated pwsh.exe path from the target system. Do not assume every installation uses C:\Program Files\PowerShell\7.

6.2 Test one workload at a time

- Run the script manually in a non-production test context
- Confirm required modules import without errors
- Confirm the application owner or vendor supports the workload under PowerShell 7.
- Compare output, error handling, logging, exit codes, and authentication behavior with the 5.1 run
- Test any direct .NET calls, COM objects, snap-ins, legacy modules, and remoting commands carefully
- Update the scheduled task, RMM component, service, or shortcut to call pwsh.exe only after the test passes
- Record the tested PowerShell version and module versions with the script

6.3 Compatibility fallback for a legacy module

On Windows, a module that will not load natively may still work through the Windows PowerShell compatibility feature:

```
Import-Module <ModuleName> -UseWindowsPowerShell
```

COMMON GOTCHA: Compatibility mode is a bridge, not proof that the module is natively compatible with PowerShell 7. Objects may be remoted and deserialized, and some commands will not behave exactly as they did in-process under 5.1. Test the real administrative workflow, not just whether Import-Module returns without complaining.

7. MIGRATE AUTOMATION SAFELY

Automation Type	What to Change	Validation
Scheduled task	Program/script: pwsh.exe; review arguments and working directory	Run on demand, confirm result code, logs, and downstream effect
RMM job	Select or call the PowerShell 7 host explicitly	Test on a pilot ring before broad targeting
Shortcut or runbook	Replace powershell.exe only after compatibility testing	Confirm the intended host opens and profile behavior is correct
Remoting	Use a PowerShell 7 endpoint only where deliberately configured	Test authentication, endpoint, serialization, and permissions

RECOMMENDATION: Do not change the system association for every .ps1 file and declare victory. Name pwsh.exe explicitly in production automation. It removes ambiguity and gives the next technician a fighting chance.

8. KEEP POWERSHELL 7 UPDATED

PowerShell 7 can receive updates through Microsoft Update when that option is enabled and organizational policy allows it. For a WinGet-managed installation, check for and apply an available update with:

```
winget list --id Microsoft.PowerShell --upgrade-available
winget upgrade --id Microsoft.PowerShell
```

When possible, update PowerShell 7 with the same method used for the original deployment. Treat minor-version changes like any other software update: test critical automation, move through a pilot ring, and then expand.

9. TROUBLESHOOTING

Symptom	Likely Cause	Corrective Action
pwsh is not recognized	New PATH has not reached the current process, or installation failed	Close the terminal, open a new one, and verify C:\Program Files\PowerShell\7\pwsh.exe.
Module imports in 5.1 but fails in 7	Desktop-only module, snap-in, or .NET dependency	Check vendor support, try a newer module, or test -UseWindowsPowerShell as a temporary bridge.
Script works manually but scheduled task fails	Task still calls powershell.exe or uses the wrong working directory/account	Inspect the action, account, arguments, PATH, working directory, and exit code.
Wrong profile or missing aliases	PowerShell 7 uses a separate profile path	Review $PROFILE in both editions and migrate only the settings that are still appropriate.
Unexpected older 7.x version starts	Multiple installs or PATH ordering	Check (Get-Process -Id $PID).Path, Get-Command pwsh -All, and $PSHOME.

10. ROLLBACK AND REMOVAL

Because Windows PowerShell 5.1 remains installed, rollback usually means pointing the affected automation back to powershell.exe while you fix the compatibility issue. If PowerShell 7 itself must be removed, use the same package-management method used to install it.

```
winget uninstall --id Microsoft.PowerShell
```

- Restore scheduled task, RMM, shortcut, or runbook references to the previously documented powershell.exe command
- Restore the prior script version from source control if the script was modified for 7.x
- Confirm the original 5.1 workflow still completes and produces the expected output
- Record the rollback reason and the failed compatibility test

COPYRIGHT AND PUBLISHING NOTICE

© 2026 Abacus Information Systems, Inc. All Rights Reserved.

This publication contains proprietary analysis, structure, commentary, recommendations, and original written content developed by Abacus Information Systems. This guide may be downloaded and used for internal administrative reference. Redistribution, modification, incorporation into another work, or republication requires prior written permission from Abacus Information Systems, Inc., except for brief quotations used with clear attribution where permitted by law.

Product information is based on identified sources and Abacus hands-on experience described in this document. Product capabilities, licensing, pricing, integrations, support, and availability may change.

Readers should validate material requirements through current documentation, written vendor confirmation, and testing before purchase or deployment.

Opinions and recommendations are the independent judgment of Abacus Information Systems. They are not warranties, certifications, legal advice, compliance determinations, or guarantees of performance. Unless explicitly disclosed, referenced vendors have not reviewed, approved, sponsored, or endorsed this publication. All product names, trademarks, service marks, and logos remain the property of their respective owners.